

如何在 Flask 應用程式使用 App Engine Memcache (模組 12)

來源：<https://codelabs.developers.google.com/codelabs/cloud-gae-python-migrate-12-memcache?hl=zh-tw>

步驟數：8

瞭解如何將 Memcache 用量新增至基本的 Python 2 Flask App Engine NDB 應用程式。

目錄

1. [1. 總覽](#)
2. [2. 背景](#)
3. [3. 設定/準備工作](#)
4. [4. 更新設定](#)
5. [5. 修改應用程式檔案](#)
6. [6. 摘要/清除](#)
7. [7. 加分題：遷移至 Python 3](#)
8. [8. 其他資源](#)

1. 總覽

[無伺服器遷移工作站系列程式碼研究室](#) (自學式實作教學課程) 和[相關影片](#)，旨在協助 [Google Cloud 無伺服器](#)開發人員完成一或多項遷移作業 (主要是從舊版服務遷移)，進而翻新應用程式。這樣做可提高應用程式的可攜性，並提供更多選項和彈性，讓您整合及存取更多 Cloud 產品，並更輕鬆地升級至新版語言。雖然一開始的重點是早期 Cloud 使用者，主要是 [App Engine](#) (標準環境) 開發人員，但本系列涵蓋範圍廣泛，也包括其他無伺服器平台，例如 [Cloud Functions](#) 和 [Cloud Run](#)，或適用於其他平台。

本程式碼研究室說明如何納入及使用 [App Engine Memcache](#)，為第 1 模組程式碼研究室的範例應用程式提供快取功能。我們會在第 12 堂教學課程中加入 Memcache 的使用方式，然後在第 13 堂[遷移至 Cloud Memorystore](#)。

未使用 Memcache 嗎？

如果您的應用程式未使用 App Engine Memcache，可以略過第 12 到 13 個單元，也可以完成這些程式碼研究室，熟悉 Memcache 遷移作業。

在接下來的研究室中

- 使用 App Engine Memcache API/程式庫
- 在基本 Python 2 Flask App Engine NDB 應用程式中新增快取

軟硬體需求

- 擁有 [Google Cloud Platform 專案](#)，且已啟用 [GCP 帳單帳戶](#)
- Python 基礎技能
- 熟悉常見的 Linux 指令
- 具備[開發及部署](#) App Engine 應用程式的基本知識
- 可正常運作的第 1 模組 App Engine 應用程式 (建議完成[程式碼研究室](#)，或從存放區複製[應用程式](#))

問卷調查

2. 背景

如要從 App Engine Memcache 遷移，請將其用法新增至現有的 Flask 和 App Engine NDB 應用程式，也就是[第 1 堂程式碼研究室](#)的成果。範例應用程式會顯示使用者最近的十次造訪記錄。如果同一位使用者重新整理瀏覽器，持續建立新的 Visit 實體並從 Datastore 擷取最近的造訪記錄並非最佳做法，因此我們將快取這些最近的造訪記錄。

如果同一位訪客造訪該網頁，系統會從快取傳回這些造訪。如果新使用者造訪網站或經過一小時，系統就會清除快取，並以最新項目取代 (更別說會註冊新的造訪)。實作這個 App Engine Memcache 整合後，我們就能在下一個 (第 13 堂) 程式碼研究室中，將其遷移至 [Cloud Memorystore](#)。

本教學課程包含下列步驟：

1. 設定/準備工作
 2. 更新設定
 3. 修改應用程式程式碼
-

3. 設定/準備工作

在進入教學課程的主要部分之前，請先設定專案、取得程式碼，然後部署基準應用程式，確保我們從可運作的程式碼開始。

1. 設定專案

如果您已完成[第 1 模組](#)程式碼研究室，建議重複使用該專案 (和程式碼)。或者，您也可以[建立全新專案](#)，或重複使用其他現有專案。確認專案已啟用 App Engine，且具備有效的帳單帳戶。

2. 取得基準範例應用程式

本程式碼研究室的先決條件之一，是必須有可運作的第 1 課範例應用程式。如果沒有，請先完成任一教學課程 (上方連結)，再繼續進行本程式碼研究室。如果您已熟悉內容，可以直接從下方的第 1 堂課程程式碼開始。

無論您使用自己的程式碼或我們的程式碼，我們都會從「模組 1」的程式碼開始。本程式碼研究室會逐步引導您完成每個步驟，最後的程式碼會與 Module 11 存放區資料夾 (FINISH) 中的程式碼類似。

- 開始：[模組 1 資料夾](#) (Python 2)
- 完成：[第 12 堂課程資料夾](#) (Python 2)
- [整個存放區](#) (複製或下載 [ZIP 檔案](#))

模組 1 STARTING 檔案的目錄 (您的或我們的) 應如下所示：

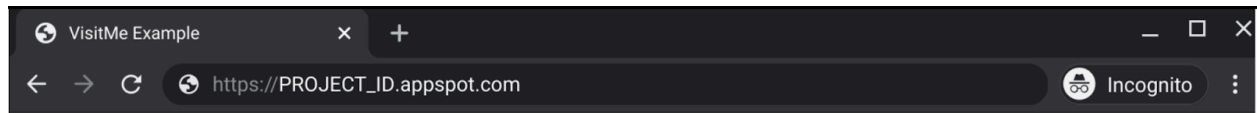
```
$ ls
README.md          main.py            templates
app.yaml           requirements.txt
```

3. (重新) 部署基準應用程式

請立即執行剩餘的準備步驟：

1. 重新熟悉 `gcloud` 指令列工具
2. 使用 `gcloud app deploy` 重新部署範例應用程式
3. 確認應用程式在 App Engine 上順利執行

成功執行上述步驟並確認網頁應用程式正常運作 (輸出內容類似下方範例) 後，即可在應用程式中加入快取功能。



VisitMe example

Last 10 visits

- Tue Mar 30 06:49:20 2021 from 2601:647:4300:5df:2ac6:3fff:fe34:13c7: Mozilla/5.0 (Macintosh; Intel Mac OS X 11_2_2) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/89.0.4389.90 Safari/537.36
 - Sun Mar 28 03:45:17 2021 from 2601:647:4300:5df:2ac6:3fff:fe34:13c7: Mozilla/5.0 (X11; CrOS x86_64 13597.94.0) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/88.0.4324.186 Safari/537.36
 - Sat Mar 27 20:31:55 2021 from 2601:647:4300:5df:2ac6:3fff:fe34:13c7: curl/7.64.0
 - Sat Mar 27 20:23:39 2021 from 2601:647:4300:5df:2ac6:3fff:fe34:13c7: Mozilla/5.0 (X11; CrOS x86_64 13597.94.0) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/88.0.4324.186 Safari/537.36
 - Fri Mar 26 04:09:23 2021 from 2601:647:4300:5df:2ac6:3fff:fe34:13c7: curl/7.64.1
 - Fri Mar 26 04:07:26 2021 from 2601:647:4300:5df:2ac6:3fff:fe34:13c7: curl/7.64.1
 - Sat Mar 20 00:17:54 2021 from 2601:647:4300:5df:2ac6:3fff:fe34:13c7: curl/7.64.1
 - Sat Mar 20 00:02:06 2021 from 2601:647:4300:5df:2ac6:3fff:fe34:13c7: Mozilla/5.0 (Macintosh; Intel Mac OS X 11_2_2) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/89.0.4389.82 Safari/537.36
 - Sat Mar 20 00:00:15 2021 from 2601:647:4300:5df:2ac6:3fff:fe34:13c7: Mozilla/5.0 (Macintosh; Intel Mac OS X 11_2_2) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/89.0.4389.82 Safari/537.36
 - Fri Mar 19 23:28:54 2021 from 127.0.0.1: Mozilla/5.0 (Macintosh; Intel Mac OS X 11_2_2) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/89.0.4389.82 Safari/537.36
-

步驟 4 · 約 1 分鐘

4. 更新設定

標準 App Engine 設定檔 (`app.yaml` 、 `requirements.txt` 、 `appengine_config.py`) 無須變更。

5. 修改應用程式檔案

由於我們只新增 App Engine API，因此不會涉及任何外部套件，也就是說，不需要更新任何設定檔（`app.yaml`、`requirements.txt`、`appengine_config.py`）。只有一個應用程式檔案 `main.py`，因此本節中的所有變更只會影響該檔案。

匯入

最重要的步驟是匯入 Memcache 程式庫 `google.appengine.api.memcache`。由於我們將快取最近一小時的造訪記錄，因此請新增一小時的秒數常數。以下是程式碼在變更前後的樣子：

BEFORE:

```
from flask import Flask, render_template, request
from google.appengine.ext import ndb

app = Flask(__name__)
```

修改後：

```
from flask import Flask, render_template, request
from google.appengine.api import memcache
from google.appengine.ext import ndb

app = Flask(__name__)
HOUR = 3600
```

新增快取功能，支援 Memcache

最重大的變更是在應用程式中新增快取功能。具體來說，我們應該快取最近的造訪記錄、檢查是否有可用的快取造訪記錄，並盡可能根據我們的計畫使用快取結果。應用程式會採取下列步驟來達成目標：

1. 設定目前的造訪並呼叫 `visitor`
2. 嘗試從快取擷取最新的 `visits`
3. 如果快取為空白，或最近一次訪客（`visits[0]['visitor']`）與目前的 `visitor` 不同：儲存最近一次造訪、擷取最近幾次造訪，並快取一小時。
4. 透過網頁範本向使用者顯示 `visits`

以下是更新前後的畫面：

BEFORE:

```
@app.route('/')
def root():
    'main application (GET) handler'
    store_visit(request.remote_addr, request.user_agent)
    visits = fetch_visits(10)
    return render_template('index.html', visits=visits)
```

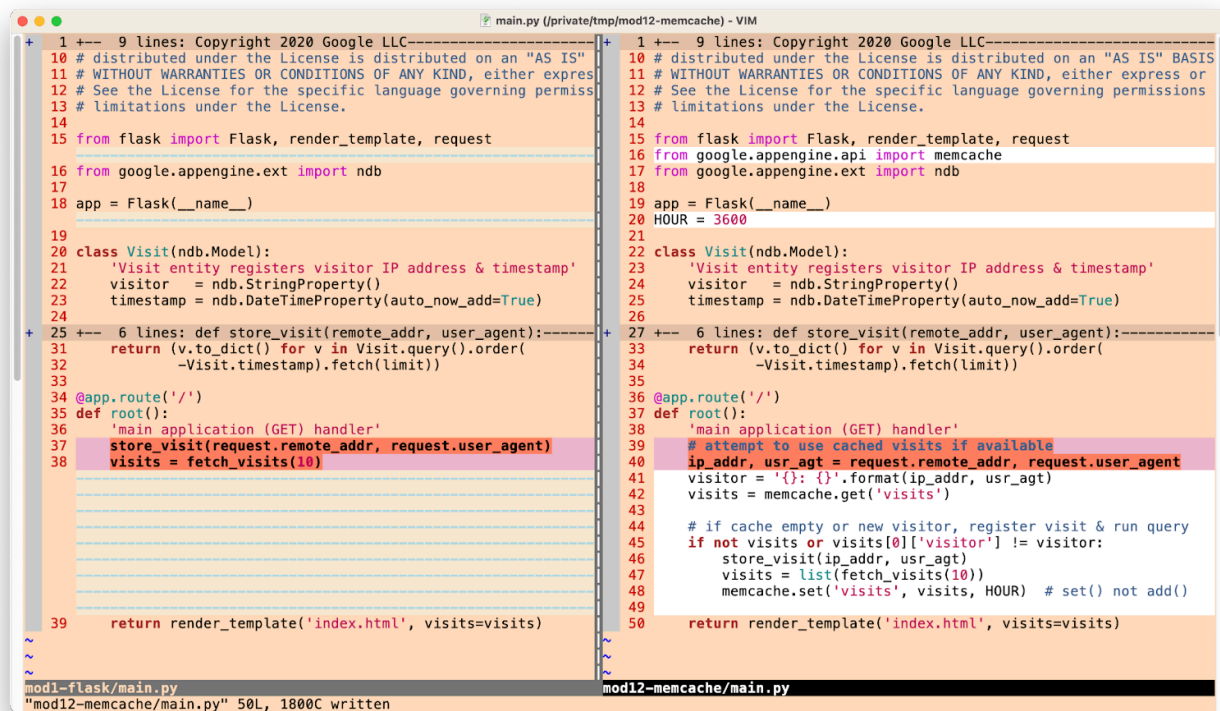

修改後：

```
@app.route('/')
def root():
    'main application (GET) handler'
    # check for (hour-)cached visits
    ip_addr, usr_agt = request.remote_addr, request.user_agent
    visitor = '{}: {}'.format(ip_addr, usr_agt)
    visits = memcache.get('visits')

    # register visit & run DB query if cache empty or new visitor
    if not visits or visits[0]['visitor'] != visitor:
        store_visit(ip_addr, usr_agt)
        visits = list(fetch_visits(10))
        memcache.set('visits', visits, HOUR) # set() not add()

    return render_template('index.html', visits=visits)
```

以下是變更內容的圖像表示法：



至此，我們已完成所有必要變更，可將 App Engine `memcache` 的使用方式新增至第 1 模組範例應用程式。現在就來建構及部署這個應用程式，看看是否能正常運作！

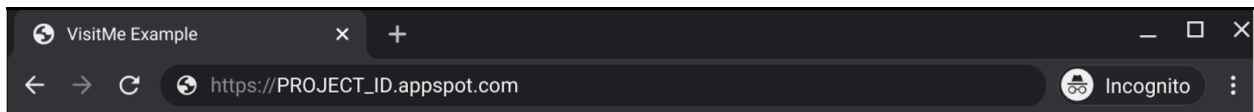
步驟 6 · 約 4 分鐘

6. 摘要/清除

本節將部署應用程式，並驗證應用程式是否正常運作，以及是否反映在輸出內容中，為本程式碼研究室畫下句點。應用程式驗證完成後，請執行任何清理步驟，並考慮後續步驟。

部署及驗證應用程式

使用 `gcloud app deploy` 重新部署應用程式，並確認應用程式是否正常運作。現在程式碼應該與「FINISH」第 12 課資料夾中的程式碼相符。輸出內容應與您先前部署的第 1 模組應用程式相同：



VisitMe example

Last 10 visits

- Tue Mar 30 06:49:20 2021 from 2601:647:4300:5df:2ac6:3fff:fe34:13c7: Mozilla/5.0 (Macintosh; Intel Mac OS X 11_2_2) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/89.0.4389.90 Safari/537.36
- Sun Mar 28 03:45:17 2021 from 2601:647:4300:5df:2ac6:3fff:fe34:13c7: Mozilla/5.0 (X11; CrOS x86_64 13597.94.0) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/88.0.4324.186 Safari/537.36
- Sat Mar 27 20:31:55 2021 from 2601:647:4300:5df:2ac6:3fff:fe34:13c7: curl/7.64.0
- Sat Mar 27 20:23:39 2021 from 2601:647:4300:5df:2ac6:3fff:fe34:13c7: Mozilla/5.0 (X11; CrOS x86_64 13597.94.0) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/88.0.4324.186 Safari/537.36
- Fri Mar 26 04:09:23 2021 from 2601:647:4300:5df:2ac6:3fff:fe34:13c7: curl/7.64.1
- Fri Mar 26 04:07:26 2021 from 2601:647:4300:5df:2ac6:3fff:fe34:13c7: curl/7.64.1
- Sat Mar 20 00:17:54 2021 from 2601:647:4300:5df:2ac6:3fff:fe34:13c7: curl/7.64.1
- Sat Mar 20 00:02:06 2021 from 2601:647:4300:5df:2ac6:3fff:fe34:13c7: Mozilla/5.0 (Macintosh; Intel Mac OS X 11_2_2) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/89.0.4389.82 Safari/537.36
- Sat Mar 20 00:00:15 2021 from 2601:647:4300:5df:2ac6:3fff:fe34:13c7: Mozilla/5.0 (Macintosh; Intel Mac OS X 11_2_2) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/89.0.4389.82 Safari/537.36
- Fri Mar 19 23:28:54 2021 from 127.0.0.1: Mozilla/5.0 (Macintosh; Intel Mac OS X 11_2_2) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/89.0.4389.82 Safari/537.36

我們只是加快了同一位使用者的體驗。重新整理時，您應該會直接從快取取得結果，這既不會建立新的造訪，也不會擷取資料儲存庫。

恭喜您完成第 12 堂程式碼研究室，為範例應用程式新增 App Engine `memcache` 服務。您現在可以選擇在額外步驟中，將這個 Python 2 應用程式移植到 Python 3。

清除所用資源

一般

如果暫時不需要使用，建議停用 App Engine 應用程式，以免產生費用。不過，如果您想進一步測試或實驗，App Engine 平台提供免付費配額，只要不超出該用量層級，就不會產生費用。這是指運算費用，但您可能也需要支付相關 App Engine 服務的費用，因此請參閱其定價頁面瞭解詳情。如果這項遷移作業涉及其他雲端服務，則這些服務會另外計費。無論是哪種情況，請視需要參閱下方的「本程式碼研究室專用」一節。

為求完整揭露，部署至 App Engine 等 Google Cloud 無伺服器運算平台時，會產生少量建構和儲存空間費用。Cloud Build 和 Cloud Storage 都有各自的免費配額。儲存該圖片會耗用部分配額。不過，你所在的區域可能沒有這類免付費層級，因此請留意儲存空間用量，盡量減少潛在費用。您應審查的特定 Cloud Storage「資料夾」包括：

- `console.cloud.google.com/storage/browser/LOC.artifacts.PROJECT_ID.appspot.com/containers/images`
- `console.cloud.google.com/storage/browser/staging.PROJECT_ID.appspot.com`
- 上述儲存空間連結取決於您的 `PROJECT_ID` 和 `* LOC *`ation，例如，如果您的應用程式託管於美國，則為「us」。

另一方面，如果您不打算繼續使用這個應用程式或其他相關的遷移 Codelab，並想完全刪除所有內容，請[關閉專案](#)。

本程式碼研究室專用

下列服務是本程式碼研究室的專屬服務。詳情請參閱各項產品的說明文件：

- App Engine Memcache 服務有兩種不同版本，各有[定價結構](#)，因此您必須追蹤與帳單相關的使用量。
- [App Engine Datastore](#) 服務是由 [Cloud Datastore](#) (Cloud Firestore Datastore 模式) 提供，這項服務也有免費方案；詳情請參閱[定價頁面](#)。

後續步驟

[第 13 單元](#)將介紹下一個可考慮的邏輯遷移作業，說明開發人員如何從 App Engine `memcache` 服務遷移至 [Cloud Memorystore](#)。這些遷移作業都是選用項目，使用者可視需求採取不同步驟，讓應用程式與時俱進。Cloud Memorystore 服務是 App Engine `memcache` 的重大升級，原因如下：

- Cloud Memorystore 並非無伺服器服務。也就是說，您必須為快取分配伺服器。Cloud Memorystore 也沒有免費方案。這兩項因素都會對費用造成重大影響。
- Cloud Memorystore 支援兩種不同的基礎儲存機制 (快取引擎)：[Redis](#) 和 [Memcached](#)。
- Cloud Memorystore (適用於 Redis) 的功能集比 App Engine Memcache 更豐富且深入。
- 如要使用 Cloud Memorystore，您必須設定 Cloud Memorystore 伺服器，並將其新增至 [Google Cloud 虛擬私有雲](#) 網路，然後讓 App Engine 應用程式使用該網路與 Memorystore 伺服器通訊。

如果您認為不需要 Cloud Memorystore 的所有功能，或擔心這項服務會影響成本，可以繼續使用 App Engine Memcache。

第 13 節之後還有許多其他可能的遷移作業，例如 Cloud NDB 和 Cloud Datastore，或是 Cloud Tasks。此外，您也可以將其他產品遷移至 Cloud Run 和 Cloud Functions。您可以在[遷移存放區](#)中找到所有這些檔案。

另一個可能的後續步驟是移植到 Python 3，下節會將此步驟視為選用步驟。

7. 加分題：遷移至 Python 3

總覽

本節包含選用的加碼內容，可將我們剛完成的[第 12 堂課應用程式](#)遷移至 Python 3。我們會先進行設定，然後再開始應用程式。

簡化 app.yaml

Python 3 執行階段的優點之一，就是可以大幅簡化 `app.yaml`。

BEFORE:

以下是單元 12 結束時 `app.yaml` 的內容：

```
runtime: python27
threadsafe: yes
api_version: 1

handlers:
- url: /*
  script: main.app
```

由於 Python 3 執行階段要求網頁架構自行進行路由作業，因此 `app.yaml` 中的所有路徑處理常式都必須變更為 `auto`。如果沒有提供任何靜態檔案，使用者可以直接移除整個 `handlers:` 區段。此外，`threadsafe` 和 `api_version` 都已淘汰。

修改後：

完成上述必要變更後，Python 3 的替代 `app.yaml` 如下所示：

```
runtime: python39
app_engine_apis: true
```

唯一需要說明的行是 `app_engine_apis: true`。2021 年，第二代執行階段開始支援舊版 App Engine 服務，但部分執行階段 (包括 Python 3) 需要額外啟動程序，才能存取 `ndb`、`taskqueue` 和 `memcache` 等 API。設定中的這一行就是為了達到這個目的。

更新 requirements.txt

您必須在 `requirements.txt` 中啟動原始 API 的另一個啟動程序，才能存取新的 App Engine SDK。

BEFORE:

以下是單元 12 結束時 `app.yaml` 的內容：

```
flask
```

修改後：

只要新增 App Engine Python SDK，您應該就會看到下列項目：

```
flask
appengine-python-standard
```

套件版本號碼

存放區中的 `requirements.txt` 檔案可能含有版本號碼。我們未在此列出這些項目，是因為這些項目會定期變更。如果未指定版本號碼，系統會使用最新版本。

刪除 `appengine_config.py` 和 `lib`

新一代 App Engine 執行階段會大幅調整第三方套件的使用方式：

- 內建程式庫是由 Google 審查並在 App Engine 伺服器上提供，可能是因為內含開發人員不得部署至雲端的 C/C++ 程式碼。這類程式庫已不再適用於第 2 代執行階段。
- 在第 2 代執行階段中，不再需要複製非內建程式庫 (有時稱為「供應商」或「自行套裝組合」)。而應列在 `requirements.txt` 中，建構系統會在部署時自動為您安裝。

由於第三方套件管理系統有所變更，因此不需要 `appengine_config.py` 檔案和 `lib` 資料夾，請將其刪除。在第 2 代執行階段中，App Engine 會自動安裝 `requirements.txt` 中列出的第三方套件。摘要：

- 不得自行組合或複製第三方程式庫，請在 `requirements.txt` 中列出這些程式庫
- 沒有 `pip install` 進入 `lib` 資料夾，也就是沒有 `lib` 資料夾週期
- `app.yaml` 中沒有內建第三方程式庫的清單 (因此沒有 `libraries` 區段)，請在 `requirements.txt` 中列出
- 應用程式不需要參照任何第三方程式庫，因此不需要 `appengine_config.py` 檔案

開發人員只需在 `requirements.txt` 中列出所有需要的第三方程式庫。

更新應用程式以使用 App Engine SDK

如上所述，Python 3 應用程式必須經過修改，才能存取 App Engine 套裝組合服務：

- 套件化 App Engine SDK (位於 `requirements.txt` 中)
- 啟用 App Engine SDK (在 `app.yaml` 中)
- 包裝 WSGI 物件 (位於 `main.py` 中)

上述已完成第一組配對，因此最後一個要求是更新 `main.py`。

BEFORE:

以下是第 12 堂課結束時的 Python 2 `main.py`：

```
from flask import Flask, render_template, request
from google.appengine.api import memcache
from google.appengine.ext import ndb

app = Flask(__name__)
HOUR = 3600
```

修改後：

如要移植 Python 3，請匯入 SDK 並包裝 Flask 應用程式物件 (SDK 包裝函式)，如下所示：

```
from flask import Flask, render_template, request
from google.appengine.api import memcache, wrap_wsgi_app
from google.appengine.ext import ndb

app = Flask(__name__)
app.wsgi_app = wrap_wsgi_app(app.wsgi_app)
HOUR = 3600
```

從 2.x 遷移至 3.x 時，開發人員需要對 Python 應用程式進行這些變更，[才能存取隨附服務](#)。如果您未使用 Flask，文件中也有 [Django](#) 和 [Pyramid](#) 範例。如果 Python 2 程式碼不是網路應用程式，移植到 Python 3 時，只要加入 SDK 套件即可。我們的應用程式程式碼原本就是為了在 Python 2 和 3 下運作而設計，因此不需要進行額外的相容性變更。

部署應用程式

完成上述變更後，即可部署更新後的範例應用程式 (在同一個 GCP 專案中，將 Python 3 版本的應用程式部署到原始 Python 2 版本上，不會發生問題)。應用程式行為應維持不變。如要比較更新後的應用程式與我們的應用程式，請參閱遷移存放區中的「[Module 12b folder](#)」。如要進一步瞭解最新執行階段 (例如 Python 3) 對 App Engine 組合式服務的支援，請參閱[功能發布公告](#)和[第 17 堂程式碼研究室課程](#)。

恭喜你完成第 12 單元的加分步驟！另請參閱[準備 Python 3 執行階段設定檔的說明文件](#)。請參閱上方的「摘要/清除」一節，瞭解後續步驟和清除作業。

8. 其他資源

以下列出其他資源，供開發人員進一步瞭解這個或相關的遷移模組，以及相關產品。包括提供內容意見回饋的位置、程式碼連結，以及您可能會覺得實用的各種文件。

程式碼研究室問題/意見回饋

如果發現本程式碼研究室有任何問題，請先搜尋問題，再提出回報。搜尋及建立新問題的連結：

- [搜尋問題](#)
- [回報新問題/提供意見](#)

遷移資源

下表提供單元 2 (START) 和單元 12 (FINISH) 的存放區資料夾連結。您也可以從[所有 App Engine Codelab 遷移作業的存放區](#)存取這些範例，並複製或下載 ZIP 檔案。

Codelab	Python 2	Python 3
Module 1	code	程式碼 (本教學課程未介紹)
單元 12 (本程式碼研究室)	code	code

線上參考資料

以下是可能與本教學課程相關的線上資源：

App Engine

- [App Engine 說明文件](#)
- [Python 2 App Engine \(標準環境\) 執行階段](#)
- [Python 3 App Engine \(標準環境\) 執行階段](#)
- [Python 2 和 3 App Engine \(標準環境\) 執行階段的差異](#)
- [Python 2 到 3 App Engine \(標準環境\) 遷移指南](#)
- App Engine [定價](#)和[配額](#)資訊
- [推出第二代 App Engine 平台 \(2018 年\)](#)
- [比較第一代和第二代平台](#)
- [對舊版執行階段的長期支援](#)
- [說明文件遷移範例存放區](#)
- [社群貢獻的遷移範例存放區](#)

Cloud Memorystore 和 Cloud Datastore

- [Cloud Memorystore 產品頁面](#)
- [Cloud Memorystore for Redis 說明文件](#)
- [Cloud Memorystore for Memcached 說明文件](#)

- [Cloud Memorystore \(for Redis\) 定價資訊](#)
- [Cloud Datastore 說明文件](#)
- [Cloud Datastore 定價資訊](#)

其他雲端資訊

- [在 Google Cloud Platform 上執行 Python](#)
- [Google Cloud Python 用戶端程式庫](#)
- [Google Cloud 「永久免費」 方案](#)
- [Google Cloud SDK \(`gcloud` 指令列工具\)](#)
- [所有 Google Cloud 說明文件](#)

影片

- [無伺服器遷移工作站](#)
- [無伺服器 Expeditions](#)
- [訂閱 Google Cloud Tech](#)
- [訂閱 Google Developers](#)

授權

這項內容採用的授權為 [Creative Commons 姓名標示 2.0 通用授權](#)。
